

Project Report Template

Advanced Analytics and Applications

Team 1



Authors:

Ivan Kamal Mehieddin (7396805)

Nils Bringewatt (7443820)

Maike Katterbach (7444323)

Vitus Grofl (7380537)

Contact: mkatter1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

1	Introduction	1
2	Data	1
2.1	Data Sources	1
2.2	Cleaning Pipeline	1
2.3	Result	2
3	Data Analysis Workflow	2
3.1	Prototyping in Notebooks	2
4	Citations and References	2
5	Conclusion	4

List of Figures

1	A plot generated in a prototyping notebook and embedded in the report.	3
2	A sample plot generated directly in the report.	3

1 Introduction

This report serves as a template for your team project. It is built using Quarto, which allows you to integrate your Python or R analysis directly into your writing.

By using this template, you ensure that your formatting (margins, line spacing, and fonts) matches the required standards for submission. The left margin is set to 4cm to allow for comments, while the other margins are 2.5cm. Line spacing is set to 1.5, and the main font is Times New Roman.

2 Data

2.1 Data Sources

We combine two data sources for the year 2025:

- **Chicago Taxi Trips** — the full set of reported taxi trips, pulled from the City of Chicago Data Portal (Socrata API, dataset `ajtu-isnz`). Because the API caps the rows per request, trips are downloaded in weekly chunks with a checkpoint file, so an interrupted download can resume without re-fetching completed weeks. The raw file contains 6,825,838 trips.
- **Weather data** — hourly temperature, precipitation, snowfall, wind and cloud cover for Chicago, from the Open-Meteo archive API, later used as exogenous features.

2.2 Cleaning Pipeline

All cleaning is done on the raw taxi export in `notebooks/00_data_loading.ipynb`; the result is saved as a Parquet file for downstream use. The pipeline removes 9.1% of trips in total. Most steps are straightforward NA/outlier filters; the two steps below need more explanation because the underlying data quirks are easy to misinterpret.

1. Duplicates and IDs. Before any other filtering, trips are deduplicated on `trip_id` (keeping the first occurrence). This has to happen first and separately from missing-ID handling: rows without a `trip_id` are set aside before deduplication, otherwise pandas would treat several trips that simply lack an ID as duplicates of each other and silently discard genuine trips.

2. Census Tract vs. Community Area. Chicago reports location at two resolutions: `census_tract` (fine-grained, $\sim 0.1 \text{ km}^2$) and `community_area` (coarse, 77 zones covering the whole city). `census_tract` is missing for $\sim 55\%$ of trips — this is not random: the city masks the tract whenever too few trips occur in it, for privacy reasons. `community_area`, in contrast, is filled for $\sim 91\text{--}97\%$ of trips, making it the more reliable field for anything that needs full spatial coverage.

A missing `community_area` (`CA = null`) means something different from a masked tract: it indicates the pickup or dropoff point lies **outside Chicago city limits**, since Chicago only assigns community areas within the city. These trips are essentially unlocatable — 95% of them also lack coordinates entirely. Rather than dropping them (they can still be valid trips for non-spatial analyses), we keep every row and instead add two indicator columns, `pickup_flag` / `dropoff_flag`, marking whether a community area is present. This lets later spatial analyses filter on the flags without losing the trips for everything else.

A second, subtler issue sits in the centroid coordinate columns: `pickup/dropoff_centroid_latitude/longitude`. It silently mix two resolutions — where a census tract exists, the coordinates are the tract centroid; where it doesn't, they fall back to the community area centroid. Comparing these coordinates directly across rows therefore mixes precision levels. We derive a clean mapping of each community area to its official centroid (using exactly the rows where the tract is absent, i.e. where the

raw coordinate *is* the CA centroid) and join it back as `pickup/dropoff_ca_centroid_lat/lon`, giving a resolution-consistent coordinate to use for any area-level aggregation (e.g. heatmaps).

3. Remaining filters (brief). Trips missing core trip values (`fare`, `trip_total`, `trip_seconds`, `trip_miles`) are dropped, as no meaningful analysis is possible without them. Negative `tips/tolls/extras` are removed (zero is legitimate — e.g. cash trips report no tip). Trips below Chicago’s legal minimum fare (\$3.25 flag-drop) or shorter than 60 seconds are treated as test/error entries and removed; trips above the 99.9th percentile of distance or duration are capped as extreme outliers. `payment_type` and `company` are left untouched — missing values remain as NA and are handled per analysis.

2.3 Result

Step	Rows removed	Share
Duplicate <code>trip_id</code>	305	0.0%
Community area filter	0 (flagged, not dropped)	0.0%
Missing core trip values	12,502	0.2%
Negative <code>tips/tolls/extras</code>	0	0.0%
Below minimum fare/duration	598,226	8.8%
Outliers above P99.9	11,709	0.2%
Total	622,742	9.1%

The cleaned dataset has 6,203,096 trips and 27 columns.

3 Data Analysis Workflow

In this section, we demonstrate how to include analysis results in your report. You can either write code directly in this `.qmd` file or reference results from a separate prototyping notebook.

3.1 Prototyping in Notebooks

You can work on your analysis in the `notebooks/prototyping.ipynb` file. When you are ready to include a result, you can use Quarto’s `embed` shortcode. This is a powerful feature that allows you to maintain a clean report while keeping your extensive prototyping code in separate notebooks.

To do this, you must label the specific cell in your notebook using `#| label: fig-some-name` (and optional `#| fig-cap: ...`). Then, you can embed it here like this:

Note that this requires your `.ipynb` notebooks to follow Quarto’s cell-metadata syntax.

In the PDF version, the code above is hidden by default to keep the report concise and focused on the results. In the HTML version, the code can be expanded using the “Code” button.

4 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like “R for Data Science” (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

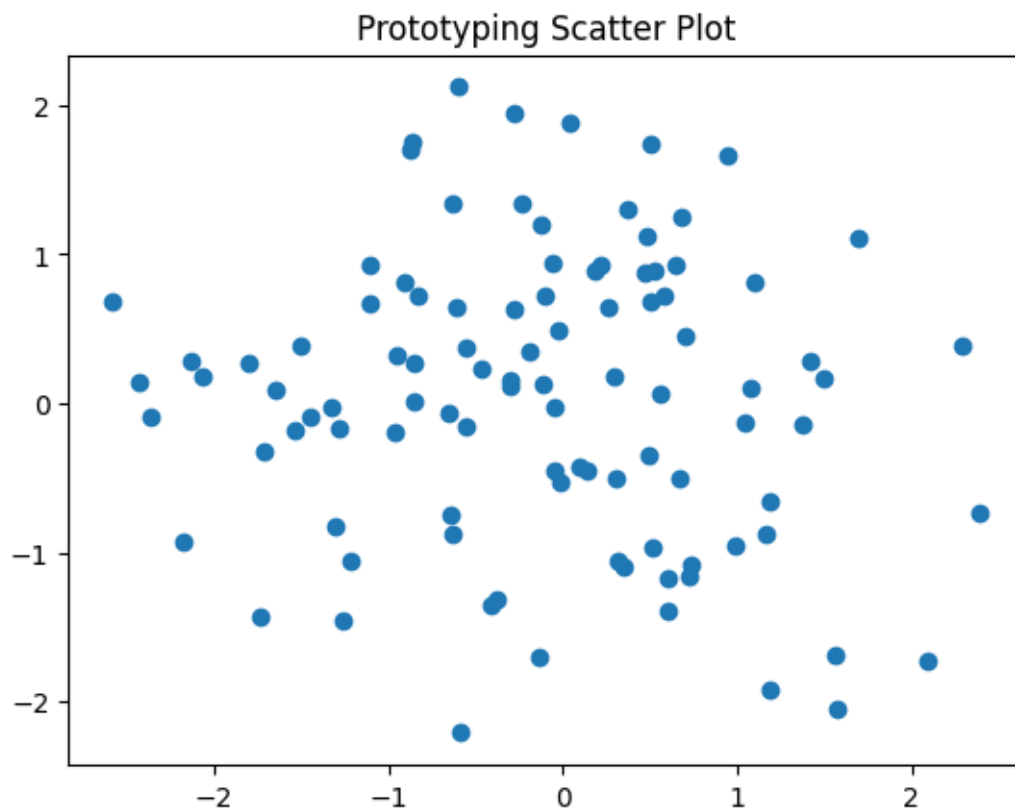


Figure 1: A plot generated in a prototyping notebook and embedded in the report.

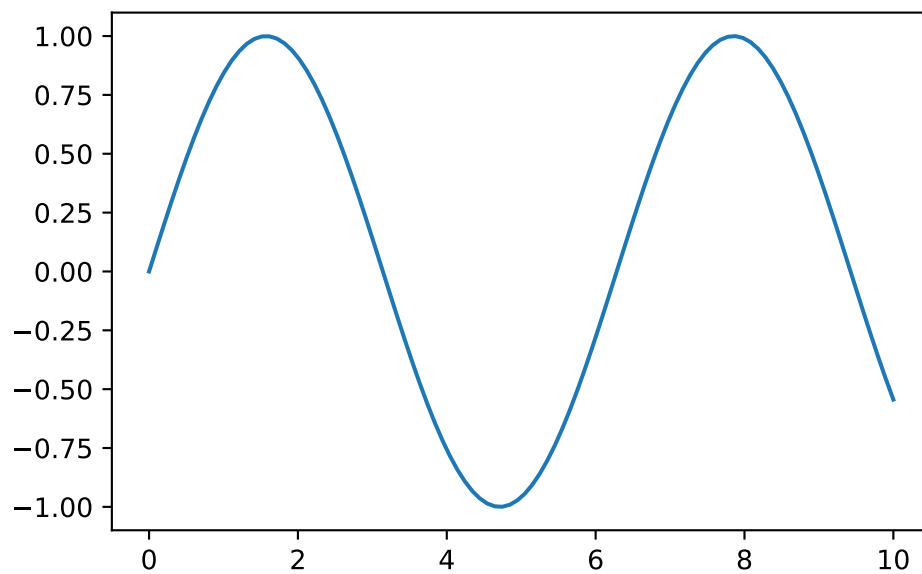


Figure 2: A sample plot generated directly in the report.

5 Conclusion

The conclusion should summarize your findings and suggest future work.

References

- Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. <https://quarto.org>
- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O'Reilly Media.